

Git Mastery Handbook

Complete Git Commands Manual for Beginners to Advanced Developers

Target Audience

- Students & Computer Science Majors
- Programmers & Coders
- Web Developers
- Software Engineers
- Open Source Contributors

Table of Contents

- 1. Introduction to Git** (Concepts, Git vs GitHub)
- 2. Git Installation & Setup** (OS Install, Config, Init)
- 3. Daily Git Workflow Commands** (Add, Commit, History, Undo)

- 4. **Working with Remote Repositories** (Push, Pull, Fetch, SSH)
- 5. **Branching & Collaboration** (Branches, Merging, PR Workflow)
- 6. **Advanced Commands & Cheat Sheet** (Stash, Revert, Best Practices)

1. Introduction to Git

What is Git?

Git is a distributed **Version Control System (VCS)** designed to track changes in source code during software development. It allows multiple developers to work together non-linearly, keeping a complete history of every change ever made.

Why Developers Use Git

Git prevents the "final_v2_really_final.txt" problem. It provides a structured timeline of changes, enables seamless team collaboration, allows you to experiment safely via branches, and acts as an ultimate backup system for your source code.

Git vs GitHub

Git	GitHub
A command-line software running locally on your computer.	A cloud-based hosting service for Git repositories.
Manages versions and history.	Provides a web interface, issue tracking, and team tools.
You don't need GitHub to use Git.	GitHub relies entirely on Git.

Core Architecture Flow

Understanding how files move through Git's states is critical. The local repository consists of three main "trees" maintained by Git:



Key Concepts Glossary

- **Repository (Repo):** A folder containing all project files and the hidden `.git` folder detailing history.
- **Commit:** A saved snapshot of your project at a specific point in time.
- **Branch:** An independent line of development. The default branch is usually `main` or `master`.
- **Merge:** The process of combining changes from one branch into another.
- **Clone:** Downloading a complete copy of a remote repository to your local machine.
- **Push:** Uploading your local commits to a remote repository.
- **Pull:** Downloading and integrating new commits from a remote repository to your local machine.

2. Git Installation & Setup

Installing Git

Before using Git, it must be installed on your operating system. Open your terminal or command prompt and use the appropriate command:

Windows

Download the installer from *git-scm.com* or use Winget:

```
winget install --id Git.Git -e --source winget  
git --version
```

Linux (Debian/Ubuntu)

```
sudo apt update  
sudo apt install git  
git --version
```

macOS

```
brew install git  
git --version
```

First Time Configuration

Git needs to know who is making commits. These settings are stored globally and applied to all repositories.

```
git config --global user.name "Your Name"  
git config --global user.email "you@example.com"
```

Check your configuration:

```
git config --list
```

Initializing a New Repository

To start tracking an existing folder with Git, navigate to the folder in your terminal and initialize it.

```
# Navigate to your project folder
cd my-project

# Initialize the Git repository
git init
```

After running `git init`, a hidden `.git` directory is created. You can verify the repository status at any time using:

```
git status
```

Troubleshooting: "Not a git repository"

If you see the error *fatal: not a git repository*, it means you are currently in a directory where `git init` has not been run, or you navigated outside of your project folder. Use `pwd` (Mac/Linux) or `cd` (Windows) to check your current directory.

3. Daily Git Workflow Commands

A typical day for a developer involves editing files, staging the changes, committing them to history, and optionally pushing them. Here is the visual flow:



Adding Files (Staging)

Before a file can be committed, it must be added to the Staging Area. This allows you to group related changes together.

```
# Stage a specific file
git add index.html

# Stage ALL changed, new, and deleted files in the current directory
git add .
```

Creating Commits

A commit permanently records the staged snapshot into the repository history.

```
git commit -m "Add login authentication feature"
```

Syntax: `git commit -m "Message"`

Common Mistake: Forgetting the `-m` flag will open a terminal text editor (like Vim) which can be confusing to exit (Type `:wq` to save and exit).

Viewing History

To view the timeline of commits made in the repository:

```
# Detailed history with author, date, and full message
git log

# Compact history (one line per commit, great for quick scanning)
git log --oneline

# Show the exact changes/diff made in a specific commit
git show [commit_hash]
```

Undoing Changes

If you make a mistake, Git provides safe ways to undo actions.

```
# Discard uncommitted changes in the working directory
git restore filename

# Unstage a file (moves it from Staging back to Working Directory)
git restore --staged filename

# Undo commits entirely (resets current HEAD to a previous state)
git reset [commit_hash]
```

Pro Tip: Always check status

Run `git status` before running `git add` or `git commit` to ensure you aren't accidentally including debug files, passwords, or compiled assets.

4. Working with Remote Repositories

Remote repositories (like GitHub, GitLab, or Bitbucket) are versions of your project hosted on the internet or a network. They allow collaboration.

Connecting to GitHub (Remotes)

If you created a repository locally and want to connect it to an empty GitHub repository:

```
# Add a new remote named 'origin'
git remote add origin https://github.com/username/repo-name.git

# Verify remote connections
git remote -v
```

Pushing and Cloning

```
# Upload local commits to the remote repository
git push origin main

# Download an entire existing repository to your computer
git clone https://github.com/username/repo-name.git
```

Fetching and Pulling Updates

When working on a team, others will push code to the remote repository. You need to download their changes.

```
# Download changes safely without modifying your working files
git fetch

# Download changes AND immediately merge them into your current branch
git pull
```

Command	Behavior	Use Case
<code>git fetch</code>	Downloads history from remote but does not merge.	Reviewing what others did before integrating it.
<code>git pull</code>	Executes <code>git fetch</code> followed by <code>git merge</code> .	Quickly updating your active branch to match the remote.

Authentication Methods: HTTPS vs SSH

To securely communicate with remote servers, Git uses two primary methods:

- **HTTPS:** Easy to set up, requires entering a Personal Access Token (PAT) instead of a password.
- **SSH (Secure Shell):** Highly recommended. Uses cryptographic keys so you don't have to authenticate on every push.

SSH Setup Example

Generate a new SSH key specifically using the modern, secure ED25519 algorithm:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

Press Enter to save to the default location. Then, output the public key to copy it:

```
cat ~/.ssh/id_ed25519.pub
```

Copy the output text and add it to your GitHub account under *Settings > SSH and GPG keys*.

5. Branching & Collaboration

Branching allows you to diverge from the main line of development and continue doing work without messing with the main codebase.

Managing Branches

```
# List all local branches (the one with * is active)
git branch

# Create a new branch
git branch feature-login

# Switch to the branch (legacy method)
git checkout feature-login

# Switch to the branch (modern method)
git switch feature-login

# Create and immediately switch to a new branch
git checkout -b feature-login
# OR modern: git switch -c feature-login

# Delete a branch locally
git branch -d feature-login
```

Merging and Resolving Conflicts

Once a feature is complete, it is merged back into the main branch.

```
# 1. Switch to the branch you want to merge INTO (usually main)
git switch main

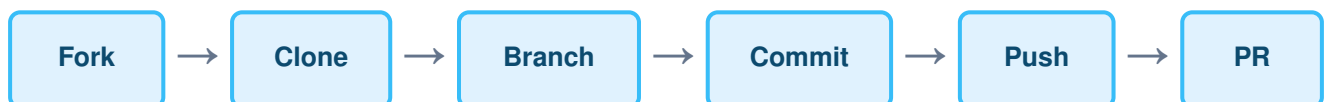
# 2. Merge the feature branch into main
git merge feature-login
```

Handling Merge Conflicts

If two branches modified the exact same part of a file, Git halts the merge and creates a **conflict**. Git will modify the affected files, adding markers like `<<<<<< HEAD` and `>>>>>> branch-name`. You must open the file, manually edit it to how it should look, save, run `git add .`, and finally `git commit` to resolve it.

Standard Collaboration Workflow

When contributing to Open Source or team projects, developers follow a specific workflow:



- **Fork:** Create a personal copy of the repository on GitHub.
- **Pull Request (PR):** After pushing your branch, you submit a PR on GitHub. This requests the project maintainers to review your code and merge it into their official repository.
- **Code Reviews:** Teammates review the PR, suggest changes, and approve it before merging to ensure code quality.

6. Advanced Commands & Cheat Sheet

Stashing, Reverting & Resetting

Stashing: Temporarily shelves your uncommitted changes so you can switch branches without committing incomplete work.

```
git stash          # Saves changes to the stash stack
git stash pop      # Applies the most recent stash and removes it
```

Reverting: The safest way to undo a commit pushed to a shared remote. It creates a *new* commit that perfectly negates the specified past commit.

```
git revert [COMMIT_ID]
```

Resetting: Moves the branch pointer backward. **Dangerous if pushed.**

```
# Undo last commit, but keep the file changes staged
git reset --soft HEAD~1

# DESTROY last commit and ALL file changes within it
git reset --hard HEAD~1
```

Releases and Tags

Tags are used to mark specific release points (e.g., v1.0).

```
git tag v1.0
git push origin v1.0
```

Useful Inspection Commands

```
git diff          # See exact line changes made to files before staging
git blame file.txt # See who wrote every single line in a file
git reflog        # A log of every action you took (a lifesaver if you mess
up)
git shortlog      # Summary of commits by author
```

Git Best Practices

- ✓ **Commit frequently:** Small, atomic commits are easier to track and debug.
- ✓ **Write meaningful messages:** "Fix typo in header" is better than "update".
- ✓ **Use branches:** Never work directly on `main`. Always branch for features/bugs.
- ✓ **Pull before pushing:** Always ensure your local repo is up-to-date to avoid conflicts.
- ✓ **Review changes:** Always use `git diff` or check status before committing.

Ultimate Git Cheat Sheet

Command	Purpose / Action
<code>git init</code>	Create a brand new local repository
<code>git status</code>	Check current status of the working tree
<code>git add .</code>	Stage all modified files for commit
<code>git commit</code>	Save snapshot of staged files into history
<code>git push</code>	Upload local commits to remote repository
<code>git pull</code>	Download and merge changes from remote
<code>git clone</code>	Copy an entire remote repository locally
<code>git branch</code>	List, create, or delete branches
<code>git merge</code>	Join two or more development histories together
<code>git stash</code>	Temporarily save modified, tracked files
<code>git reset</code>	Undo changes / move branch pointer backward
<code>git revert</code>	Create a new commit that reverses a previous commit

"Git is not just a tool; it is the foundation of modern software development."